A Nested Basket Model for the dunnhumby Complete Journey Data

Specification, inference, sampling, and an empirical study

Abstract

We develop a model of supermarket baskets designed to answer price counterfactuals and to generate synthetic shopping data. The model separates three decisions that a single-choice model conflates — whether a household buys from a category, which item it then picks, and how much of it — and estimates the coupling between them rather than assuming it. Two commitments organise the work. First, every component is introduced to answer a measurement taken on the data beforehand, and Section 3 reports those measurements. Second, every claim is a number on held-out weeks, and where a number is not comparable across settings we say so. The fitted model beats the strongest simple baseline by +0.342 nats, its price coefficient retains 0.0% of its value when the price panel is scrambled before fitting, and baskets it generates carry 87% of the real price elasticity. Along the way we report a negative result and its resolution: eleven changes to the model's interaction term failed to make generated baskets reproduce which items co-occur, and a change to the *negative sampling distribution* succeeded.

Contents

1 Introduction

1.1 Main idea

A retailer cutting one price wants to know what happens to the whole basket. Three things can happen and they are economically distinct: households that were not going to buy the category at all now do (*incidence*); households already buying the category switch to the discounted item (*allocation*); and households buying that item take more of it (*quantity*). A model that only ranks items cannot tell these apart, and the difference between them decides whether a promotion grows a category or merely cannibalises it.

Our model gives each of these its own likelihood term, sharing one item utility u_{ijt}. The coupling between incidence and allocation is a single estimated parameter per category, κ_c, with a standard nested-logit reading: at $\kappa = 1$ category volume is whatever the items sum to, at $\kappa = 0$ a price cut only moves share, and above 1 the category expands more than proportionally.

A second idea runs through the paper. The purpose of the model is partly to *generate* data — synthetic baskets on which a pricing or couponing policy could be learned — and generation is a far more demanding test than prediction. A model can score well at "which item completes this basket" while producing synthetic baskets that look nothing like real ones. We therefore evaluate the generator on five model-free, distributional checks (Section 8.10), and most of what we learned came from those rather than from the likelihood.

1.2 Main results

1. **The three margins are separately identified and they are not equal.** A price cut splits 83% allocation, 5% incidence, 12% quantity (Section 8.8). The quantity margin is invisible to a binary-purchase model and is independently corroborated by a model-free elasticity measured before fitting.

2. **The price response survives a structural placebo.** Refitting on a scrambled price panel retains 0.0% of the coefficient while leaving κ and the incidence likelihood untouched (Section 8.7).

3. **Generated baskets carry 87% of the real price elasticity** when the elasticity is re-estimated from the synthetic data with the same estimator used on the real data (Section 8.10).

4. **Matching means is not matching distributions.** Basket size agrees with real data to a hundredth on the mean and sits 0.30 away in total variation.

5. **The model improves exactly where prices move.** On the decile of held-out purchases with the largest price deviation it *gains* $+0.121$ nats while two learned baselines without a price term each lose about 0.02 (Section 8.5).

6. **It works without a basket, with one caveat.** Scored cold, with no cart at all, it reaches -2.1385 against -2.5230 for the best basket-free baseline. But a cart holding exactly one item is *worse* than an empty one unless the context is rescaled (Section 8.6).

7. **A negative result with a resolution.** Generated baskets initially reproduced the right *amount* of co-purchase structure on the wrong *pairs*. Eleven changes to the interaction term — symmetric and asymmetric, tied and untied, mean and sum pooled, with learnable scales, category contexts, a free pairwise matrix, and SHOPPER's sequential likelihood and one-vs-each loss — left the per-pair rank correlation at zero. Making the *negatives* harder moved it to $+0.091$ (Section 8.13).

1.3 Relation to prior work

The model is a descendant of SHOPPER [1] and of the nested factorization of Donnelly et al. [2]. From SHOPPER we take the item-attribute factorisation, the per-household price sensitivity as a low-rank bilinear form, and week effects as a seasonality control; Section 8.13 also adopts its group-biased negative sampling. We depart from it in three ways: we add an explicit category-incidence stage with an estimated nesting coefficient, we model units rather than binary purchase, and we condition on the store. Section 9 discusses this in detail.

2 Data

The dunnhumby "Complete Journey" dataset: two years of transactions from one grocery chain. Construction is in `scripts/22_basket_data.py`.

households N	2,066	(basket, item) rows	1,566,063
items J	5,455	baskets	199,345
categories C	188	train / val / test rows	1,228,695 / 134,665 / 202,703
sub-commodities S	758	units > 1	22.3% of rows, 42.6% of units
stores	115	assortment density	57.2%
days D	712	median repurchase gap	28 days

Filters. An item needs at least 100 purchase lines, which is statistical support for its own parameters; a household needs 20–300 trips. Nothing is dropped to protect a modelling assumption — in particular no basket is discarded for containing two items from one category, which is the filter that one-item-per-category models require and which removes most of the data.

Splits are temporal. Train is week < 83, validation 83–90, test ≥ 91. A model whose purpose is to answer "what happens next week" must be scored on later weeks; a random split would let it see each household's future. Held-out rows whose item or household never appears in training are dropped rather than scored as a cold start.

Prices. The model sees a within-item log price deviation,

$$\Delta \log p_{jst} = \Delta \log p_{jt} + d_{jsw}, \tag{1}$$

built in six steps set out in Section 2.2. Deviating within item means the item's price *level* is absorbed by its own intercept, so what identifies the price response is an item moving against its own normal price rather than expensive items differing from cheap ones.

2.1 Notation

symbol	meaning	symbol	meaning
i	household, $1..N$	N	2,066 households
j, k, m	item, $1..J$	J	5,455 items
c	category, $1..C$	C	188 categories
t	day, $1..D$	D	712 days in the panel
w	week-of-year, 1..52	s	store, 1..115
S	the set of items in a basket	n	$\lvert S \rvert$, basket size
B	the set of categories bought on a trip	n_c	items of category c the store carries
R	purchase rows in a training batch	C_r	candidate set for row r
sub(j)	item j's sub-commodity, of 758	τ_{ijt}	days since i last bought sub(j)
π_j	within-category choice probability	λ	a Poisson rate

2.2 How the price series is built

The price entering Eq. 1 is the end of a chain, and each step is a choice.

Step 1: which price. A dunnhumby transaction line records four fields we need: `SALES_VALUE` (what the retailer banked), `QUANTITY`, `COUPON_MATCH_DISC` (the retailer's match of a manufacturer coupon), and `RETAIL_DISC` (the loyalty-card discount). Three different prices can be reconstructed and they answer different questions. We use the **loyalty price**, what a card holder faces at the shelf:

$$\text{price}_\ell = \frac{\texttt{SALES\_VALUE} - \texttt{COUPON\_MATCH\_DISC}}{\texttt{QUANTITY}}. \tag{2}$$

Subtracting the coupon match removes the part of the receipt the manufacturer refunded, which is not a price the shopper responded to. We do *not* use the paid price (which adds `COUPON_DISC` back), because two shoppers at the same shelf with different coupons would then face different "prices" for the same item on the same day, and the model has one price per (item, day).

Step 2: one number per item-day. price_{jt} is the *median* loyalty price over all purchase lines of item j on day t. A median rather than a mean because a single mis-keyed quantity produces a wild per-unit value.

Step 3: fill the gaps. Only 24.7% of the $J \times D$ item-day grid is directly observed. Missing days are filled by carrying the last observed price forward, then the first observed price backward for days before an item's first sale. This assumes a posted price persists until it is next seen to change, which is what a shelf tag does.

Step 4: log and deviate.

$$\log p_{jt} = \log \max(\text{price}_{jt}, 10^{-3}), \qquad \Delta \log p_{jt} = \log p_{jt} - \frac{1}{D}\sum_{t'=1}^{D} \log p_{jt'}, \tag{3}$$

where the mean runs over **all** $D = 712$ **days of the panel**, filled days included, so it is the item's average posted price over the two years rather than an average over the days it happened to sell.

Step 5: the store deviation. For each (item, store, week) actually observed,

$$d_{jsw} = \log \tilde{p}_{jsw} - \log \tilde{p}_{jw}, \tag{4}$$

where $\tilde{p}_{jsw}$ is the median loyalty price of item j at store s in week w and $\tilde{p}_{jw}$ the median over the whole chain that week, kept only where $|d_{jsw}| > 10^{-6}$. There are 244,880 such cells, 0.53% of the (item-week $\times$ store) grid, so the overwhelming majority of lookups return $d_{jsw} = 0$ and fall back to the chain price. That is deliberate: imputing a store price from nothing would invent variation the data does not contain.

Step 6: availability. $\text{carried}[j, s]$ is true if store s sold item j *at any point* in the two years. The threshold is one sale, not three: an item a store ever sold was by definition available there, and a threshold of three marked genuine purchases as unavailable, which set the inclusive value to -10^9 for categories a household demonstrably bought from. The mask covers 57.2% of the item $\times$ store grid. dunnhumby has no stock-out feed, so this is a proxy and remains one.

3 What the data says

Every measurement in this section was taken before any model was fitted, using counts and least squares only. Each subsection ends with the design consequence, and Section 7 collects those consequences into a table. The code is `scripts/29_demand_eda.py` and `scripts/30_household_eda.py`.

3.1 Demand responds to price

Form a panel with one row per (item, week). Let b_{jw} be purchase lines and T_w the number of baskets chain-wide that week; put $y_{jw} = \log \frac{b_{jw}+0.5}{T_w}$, where the 0.5 keeps zero-purchase weeks in the panel, and let x_{jw} be the mean log price. The elasticity is a within-item least-squares slope, i.e. after removing each item's own mean from both sides:

$$\hat{\varepsilon} = \frac{\sum_{j,w}(x_{jw} - \bar{x}_j)(y_{jw} - \bar{y}_j)}{\sum_{j,w}(x_{jw} - \bar{x}_j)^2}, \qquad \text{556,410 item-weeks.} \tag{5}$$

outcome	elasticity	interpretation
log buyers per trip	−0.792	whether a household buys the item at all
log units per trip	−0.945	total units moved
log units per buyer	−0.235	how much a buyer takes, given that they buy

Of the total units response, 25% comes from existing buyers taking more. **Consequence:** a binary-purchase model discards a quarter of the price response, so quantity needs its own likelihood term.

3.2 A price cut moves three margins, and they can be measured separately

Units from a category factor exactly,

$$\frac{\text{units}}{\text{basket}} = \underbrace{\frac{\text{visits}}{\text{baskets}}}_{\text{incidence}} \cdot \underbrace{\frac{\text{lines}}{\text{visit}}}_{\text{breadth}} \cdot \underbrace{\frac{\text{units}}{\text{line}}}_{\text{depth}}, \tag{6}$$

so in logs the three elasticities must sum to the total. Estimating each by Eq. 5 on a (category, week) panel of 15,309 cells:

margin	elasticity	
incidence	−0.675	
breadth	−0.092	
depth	−0.158	
sum	−0.925	against −0.945 from Eq. 5 on the item panel

The two agree to 0.021 across different panels, different units of analysis and no shared code. About 73% of the entire units response is households deciding to enter a category at all. **Consequence:** incidence is the dominant margin and must be modelled explicitly, not folded into item choice.

3.3 Promotions: an event study

Take every item-week where log price fell by at least 0.15 against the previous week (37,132 events over 4,712 items) and normalise demand by the item's own mean. Events are not independent: 57.7% of consecutive cuts for the same item fall less than seven weeks apart, so we also report the 20,725 *isolated* events with no other cut within ±3 weeks. Neither 1.0 nor the pre-period is a valid baseline — the former averages over the promotions themselves, the latter is selected to be expensive by the event definition — so we use a *quiet week*, an item-week more than three weeks from any cut of that item. There are 368,173, running at 0.896 of the item's mean demand at essentially its normal price.

weeks from cut	demand	price (log dev.)	demand ÷ quiet
−3	0.925	+0.050	1.03
−1	0.870	+0.123	0.97
0	**2.136**	−0.194	**2.38**
+1	1.337	−0.139	1.49
+3	1.065	−0.003	1.19
quiet	0.896	+0.011	1.00

Three readings. *Demand does not ramp before a cut*: the pre-period sits within 3% of quiet, so the spike is not partly its own cause — which is what licenses reading the price coefficient causally, and is tested directly in Section 8.7. *The promotion week is much steeper than elasticity*: from −1 to 0, price moves −0.317 and demand +0.898 in logs, an implied slope of −2.83, which is 3.4× the −0.792 measured on ordinary variation — a promoted price arrives with display and feature that the price variable does not carry. *Most of the tail is the discount still running*: at +1 price is still 0.139 below normal, and by +3, when price has returned, demand is 1.19 of quiet. No offset dips below quiet, so no stockpiling is visible within three weeks.

3.4 Households differ, and by how much can be measured

Taste. Split each household's purchases in half by date, count purchases per category in each half to get two vectors in $\mathbb{R}^{188}$, scale each to unit length, and take the dot product. Against *all* 4,266,290 cross-household pairs:

own two halves	**0.7838**
two different households	0.4710
ratio	**1.66**×

A stranger already scores 0.4710 because everyone buys milk and bread; the gap is what is personal. **Consequence:** a per-household taste vector θ_i is justified.

Price sensitivity is real but poorly measured. Regress log units on log price per household, within (household, item), on pairs bought three or more times. Across 1,640 households the median is −0.169 with p10–p90 −0.427–−0.016. But spread alone proves nothing — noisy estimates are spread out even when the truth is identical — and measuring each household twice on disjoint halves gives a split-half correlation of only +0.236 on 1,374 households.

The reason is visible in the estimator, which divides by the price variation that household actually saw: 18.6% of qualifying rows come from a (household, item) pair whose price never moved. Splitting households by how much movement they had:

	spread of slopes (sd)	share positive
little price movement	0.226	11.5%
much price movement	0.129	3.5%

Where prices moved, estimates are tighter and almost all negative. **Consequence:** per-household price sensitivity is justified but must be low-rank rather than free, and a targeting policy should weight by how much is known about each household.

3.5 Stores are not interchangeable

15.8% of store-item-weeks sit more than a cent from the chain price, and the median store ever sold only 63% of the catalogue. **Consequence:** scoring an item a store never stocked as a *rejected* alternative is a specification error; it was not available to reject.

3.6 Base rates the model must reproduce

Categories per basket 6.12 of 188 (3.25%); items per basket 7.86; units per basket 10.64; distinct items per purchased category 1.284. These are the quantities Section 8.10 holds the generator to.

4 The model

A trip is one household i shopping at store s on day t, in week-of-year w. The model answers four questions about that trip, in this order:

1. For each of the C categories — does the household buy from it at all?
2. For a category it buys from — how many *different* products does it take?
3. Which products?
4. How many units of each?

Section 4.1 builds one quantity that all four answers share. Section 4.2 gives the four answers. Section 4.3 explains the one term that ties question 1 to questions 2–4, and what its coefficient means. Section 4.4 covers the term that lets items in a basket influence each other. Section 4.5 multiplies the four answers into the probability of a whole basket and says how the training objective differs from it.

4.1 The shared quantity: how appealing item j is

Everything rests on one number per (household, item, trip), which we call the item's utility:

$$\begin{aligned} u_{ijt} = \underbrace{\lambda_j}_{\text{popular?}} + \underbrace{\theta_i^\top \alpha_j}_{\text{their kind of thing?}} + \underbrace{\rho_j^\top \bar{\alpha}_j(S)}_{\text{fits the cart?}} - \underbrace{(\gamma_i^\top \beta_j)\,\Delta \log p_{jst}}_{\text{expensive today?}} \\ + \underbrace{\eta_j^\top x_{ijt}}_{\text{due a repurchase?}} + \underbrace{\mu_j^\top \delta_w}_{\text{in season?}} + \underbrace{\zeta_j^\top \xi_s}_{\text{this store's kind of thing?}} \end{aligned} \tag{7}$$

Each term is a dot product of two learned vectors, which is a compact way of writing a large table of interactions with few numbers. $\theta_i^\top \alpha_j$ is an example: rather than storing one taste value for every household–item pair ($N \times J$ numbers), each household gets a vector θ_i, each item a vector α_j, and their dot product supplies the pair. The same device gives every household its own price sensitivity $\gamma_i^\top \beta_j$ instead of one shared slope.

The price input. $\Delta \log p_{jst}$ is how far today's price sits from that item's own normal price, in logs (Section 2.2). It enters with a minus sign, so a *positive* $\gamma_i^\top \beta_j$ means the household buys less when the price is above normal.

The memory input. x_{ijt} describes how long it has been since this household last bought something of this kind. Let τ be the days since household i last bought item j's sub-commodity; then

$$x_{ijt} = \big[\ \mathbb{1}\{\text{never before}\},\quad e^{-\tau/7},\quad e^{-\tau/g_{\mathrm{sub}(j)}},\quad \log(1+\tau)/\log 100\ \big]^\top, \tag{8}$$

with the last three set to zero if there is no earlier purchase. The three decays let the model express "bought it yesterday", "about due", and "long gone" separately, and $g_{\mathrm{sub}(j)}$ is that sub-commodity's median repurchase gap so that "due" means a week for milk and months for shampoo.

4.2 The four answers

Write $\mathcal{C}_s(c)$ for the items of category c that store s carries. All four answers are read off u_{ijt}.

1. Is category c bought? A coin flip whose bias depends on the household, the season of its last purchase, and how good the category looks today:

$$y_{ct} \sim \mathrm{Bernoulli}\big(\sigma(\eta_{ict})\big), \qquad \eta_{ict} = \underbrace{c_c^0 + c_i^{u\top} c_c^c + c_c^{s\top} x_{ict}}_{\text{household and timing}} + \underbrace{\kappa_c\big(IV_{ict} - \overline{IV}_c\big)}_{\text{how good the items are}}\ , \tag{9}$$

with $\sigma(x) = 1/(1+e^{-x})$ turning the score into a probability. Term by term:

symbol	shape	what it is
y_{ct}	scalar $\in \{0,1\}$	1 if the household buys anything from category c on this trip
c_c^0	one per category	how often c is bought at all: bread often, motor oil rarely
c_i^u	K per household	what kinds of category this household leans toward
c_c^c	K per category	where category c sits among those kinds
$c_i^{u\top} c_c^c$	scalar	their match: this household's appetite for this category
x_{ict}	4 numbers	the recency basis of Section 4.1, for this category
c_c^s	4 per category	how strongly recency matters for this category
κ_c	one per category	how much the items' appeal feeds into buying the category
IV_{ict}	scalar	how good the category's items look today (Section 4.3)
$\overline{IV}_c$	one per category	a constant subtracted from IV, fixed once during fitting (Section 5.3)

So the first three terms answer "would this household buy this category on a normal day?" and the fourth adjusts for what the category is worth *today*, given its prices and what the store has.

What x_{ict} actually contains, and why it is wrong. The recency basis of Section 4.1 is defined per *sub-commodity*. A category contains several, so a category-level recency has to be constructed somehow. The code does this (`27_nested_basket.py:425`):

1. take the item in category c with the lowest `item_id`, which is the lowest `PRODUCT_ID` — an arbitrary ordering with no meaning;

2. read off that one item's sub-commodity;

3. set τ = days since household i last bought *anything in that sub-commodity*;
4. return $x_{ict} = \left[\mathbb{1}\{\text{never}\},\ e^{-\tau/7},\ e^{-\tau/g_{\text{sub}}},\ \log(1+\tau)/\log 100\right]^{\top}$.

So it is not a category-level quantity at all. It is the recency of one arbitrarily chosen sub-commodity, standing in for the category. Concretely:

category	items	subs	sub-commodity actually used	% of items
BAKED BREAD/BUNS/ROLLS	141	14	HOT DOG BUNS	**8.5%**
BAG SNACKS	153	12	PRETZELS	12.4%
CHEESE	176	11	NATURAL CHEESE CHUNKS	19.9%
FRZN MEAT/MEAT DINNERS	201	7	PREMIUM ENTREES	21.9%
SOFT DRINKS	225	14	2 LITER BOTTLE, CARBONATED	30.7%
YOGURT	148	2	YOGURT NOT MULTI-PACKS	76.4%

For bread, the model's answer to "is this household due to buy bread?" is "have they recently bought hot dog buns?". Across all 188 categories the median chosen sub-commodity covers 50% of its category's items, and in 21% of categories it covers under 20%.

This is a defect, not a modelling choice, and we record it as one (Section 10). Its effect is bounded by what the term is worth: $c_c^{s\top} x_{ict}$ sits inside the incidence score alongside the household and inclusive-value terms, and the state ablation in Section 8.3 bounds the whole recency mechanism — item-level and category-level together — at 0.071 nats. A correct version would use the days since the household last bought *any* item in the category, which is a change to the data build rather than the model; we have not made it, so every number in this paper uses the arbitrary-sub-commodity version.

2. How many different products, given it is bought?

$$k_{ct} - 1 \sim \text{Poisson}\big(e^{z^b_{ict}}\big), \qquad z^b_{ict} = b^0_c + b^u_i - b^p_c\,\overline{\Delta \log p}_{ct}, \tag{10}$$

Subtracting 1 and adding it back means $k \geq 1$: a bought category contains at least one product, and the Poisson only models how many *extra* ones.

symbol	shape	what it is
k_{ct}	integer ≥ 1	how many different products are taken from category c
b^0_c	one per category	the typical breadth of this category — one yoghurt or four
b^u_i	one per household	whether this household buys broadly or narrowly, overall
b^p_c	one per category	how much a category-wide promotion widens the selection
$\overline{\Delta \log p}_{ct}$	scalar	average price deviation over the category's stocked items

The minus sign on b^p_c means a positive b^p_c widens the basket when prices fall.

3. Which products? The k_{ct} products are drawn, without replacement, in proportion to e^u among the items the store carries — so an item twice as appealing is twice as likely, and an item the store does not stock cannot be chosen at all:

$$P(j) = \frac{e^{u_{ijt}}}{\sum_{j' \in \mathcal{C}_s(c)} e^{u_{ij't}}}, \qquad j \in \mathcal{C}_s(c). \tag{11}$$

4. How many units of each chosen product?

$$q_{jt} - 1 \sim \text{Poisson}\big(e^{z_{ijt}}\big), \qquad z_{ijt} = q^0_j - (\gamma^{q\top}_i \beta^q_j)\,\Delta \log p_{jst} + \eta^{q\top}_j x_{ijt}, \tag{12}$$

Again $q \geq 1$: a chosen product is bought at least once, and the Poisson models the extras.

symbol	shape	what it is
q_{jt}	integer ≥ 1	units of product j
q^0_j	one per item	how many people usually take: one ketchup, six yoghurts
γ^q_i	K_p per household	willingness to stock up when something is cheap
β^q_j	K_p per item	how stockpile-able this item is
η^q_j	4 per item	how time since last purchase changes the amount

Note the price coefficient here, $\gamma^{q\top}_i \beta^q_j$, is *separate* from the one in Eq. 7. That is deliberate: the data says whether a household buys an item and how much of it respond to price at different rates (Section 3.1), and sharing one coefficient would assume away the difference.

4.3 The term that ties them together, and what κ means

The four answers would be independent were it not for one term in Eq. 9. The *inclusive value*

$$IV_{ict} = \log \sum_{j \in \mathcal{C}_s(c)} e^{u_{ijt}} \tag{13}$$

collapses a whole category into one number: how appealing its items are, right now, at this store, at today's prices. If everything in the category goes on offer, every u_{ijt} rises and so does IV. The coefficient κ_c decides how much of that reaches the decision to buy the category at all.

This is the model's answer to the retailer's question. A price cut on one item raises IV a little; κ_c says whether that turns into new category buyers or merely reshuffles Eq. 11:

$\kappa_c = 0$	incidence ignores the items completely. A price cut moves share inside the category and nothing else — pure cannibalisation.
$\kappa_c = 1$	the reference point (see below): the category grows exactly as much as its items gained, and no more.
$\kappa_c > 1$	the category grows more than its items gained — the promotion pulls in trips.

Why $\kappa = 1$ is the reference point. Suppose instead of a coin flip we modelled the total units a household takes from category c as a count $Q \sim \text{Poisson}(e^{a_{ic}+\kappa_c IV_{ict}})$, split across the category's items in proportion to e^u. A Poisson total split by fixed proportions gives independent Poisson counts per item, with rates $e^{a_{ic}+\kappa_c IV} \cdot e^{u_{ijt}-IV}$, because the proportion for item j is exactly $e^{u_{ijt}-IV}$ by Eq. 13. That is

$$q_{ijt} \sim \text{Poisson}\big(e^{\,a_{ic}+(\kappa_c-1)IV_{ict}+u_{ijt}}\big), \tag{14}$$

and at $\kappa_c = 1$ the inclusive value disappears: each item's rate depends only on its own u, so the category's total is whatever its items happen to add up to. That is the sense in which $\kappa = 1$ means "no expansion", and it is why we parameterise $\kappa_c = \text{softplus}(\kappa^{\text{raw}}_c)$ starting at exactly 1.0: the data must push it off the neutral point in one direction or the other.

Eq. 14 is a device for reading κ, not a second model. The fitted model is Eqs. 9–12, in which the inclusive value appears once, in the incidence term.

4.4 Letting items in a cart influence each other

The remaining term in Eq. 7 is $\rho_j^\top \bar{\alpha}_j(S)$, where S is the set of items in the basket. For item j in a basket of n items, the context is

$$\bar{\alpha}_j(S) = \frac{1}{n-1}\Big(\sum_{k\in S}\alpha_k - \alpha_j\Big), \qquad \bar{\alpha}_j(S) := \mathbf{0} \text{ when } n = 1. \tag{15}$$

Why this is an average over the other items. Item j is itself in the basket, so $\sum_{k\in S}\alpha_k$ already includes α_j; subtracting it leaves the sum over everything else,

$$\sum_{k\in S}\alpha_k - \alpha_j = \sum_{k\in S,\, k\neq j}\alpha_k, \qquad \text{so} \qquad \bar{\alpha}_j(S) = \frac{1}{n-1}\sum_{k\in S,\, k\neq j}\alpha_k. \tag{16}$$

S has n items and one of them is j, so there are $n-1$ others; dividing by $n-1$ makes this their average. When $n = 1$ there are no others, $n-1 = 0$, and the term is set to zero.

Why the interaction is an average of pairwise affinities. A dot product is linear, so the constant comes out and $\rho_j^\top$ goes inside the sum:

$$\rho_j^\top \bar{\alpha}_j(S) = \frac{1}{n-1}\sum_{k\in S,\, k\neq j}\rho_j^\top \alpha_k. \tag{17}$$

Each $\rho_j^\top \alpha_k$ is one number saying how much having k in the cart helps or hurts j: positive means they go together, negative means they compete. The term is the average of those over everything else in the cart. For a cart $\{j, k, l, m\}$ it is $\frac{1}{3}(\rho_j^\top\alpha_k + \rho_j^\top\alpha_l + \rho_j^\top\alpha_m)$.

Eq. 15 is what the code computes, because the basket sum is accumulated once and each item's context follows from one subtraction. The data records no order within a receipt, so S is a set: the context is the whole cart, never a prefix.

Tied or untied. Setting $\rho_j = \alpha_j$ makes the interaction $\alpha_j^\top\alpha_k$, which is symmetric and largest when two items are *alike*. That forces co-purchase structure into α, which is what makes the embedding recover product categories (Section 8.9), but it cannot say that two *different* items belong together. Leaving ρ free allows that, and allows negatives for items that compete. Section 8.12 measures which is better; the tied form is the default.

What distribution over baskets this implies. Write b_j for every part of Eq. 7 that does not involve other basket items. For baskets of a fixed size n, define a score for the whole basket,

$$E(S) = \sum_{j\in S} b_j + \frac{1}{n-1}\sum_{\{j,k\}\subset S}\alpha_j^\top\alpha_k, \tag{18}$$

where $\{j,k\}\subset S$ runs over each unordered pair once, and let higher-scoring baskets be more likely, $P(S) \propto e^{E(S)}$. Now swap one item: replace $j \in S$ by some $m \notin S$, giving S'. The size does not change, so the constant $\frac{1}{n-1}$ is the same on both sides. Every item other than j and m contributes the same b to both and cancels; every pair not involving j or m likewise cancels. What is left is the pairs j had with the rest, removed, and the pairs m gains with that same rest:

$$E(S') - E(S) = (b_m - b_j) + \alpha_m^\top\bar{\alpha}_j(S) - \alpha_j^\top\bar{\alpha}_j(S) = u_m - u_j. \tag{19}$$

That is exactly the difference Eq. 11 scores when choosing between m and j. So the model's answer to "what goes in this slot, given the rest of the cart" is the conditional of $P(S) \propto e^{E(S)}$ at fixed basket size — which is what makes the Gibbs sampler of Section 6 legitimate, and is checked numerically in Section 8.2.

4.5 The probability of a whole basket

Multiplying the four answers together gives what the model says about a trip. Write

$$\mathcal{B} = \Big(\underbrace{y_1..y_C}_{\text{which categories}}, \underbrace{(k_c)}_{\text{how many products each}}, \underbrace{(J_c)}_{\text{which products}}, \underbrace{(q_j)}_{\text{how many units each}} \Big) \tag{20}$$

for a basket, where J_c is the set of k_c products taken from category c. Prices, the store, the day and the household are given, never generated. Then

$$P(\mathcal{B}) = \prod_{c=1}^{C} \underbrace{\sigma(\eta_{ict})^{y_c}\big(1-\sigma(\eta_{ict})\big)^{1-y_c}}_{\text{Eq. 9: bought or not}} \cdot \prod_{c:\, y_c=1} \Big[\underbrace{P(k_c)}_{\text{Eq. 10}} \underbrace{P(J_c \mid k_c)}_{\text{Eq. 11}} \prod_{j\in J_c} \underbrace{P(q_j)}_{\text{Eq. 12}} \Big]. \tag{21}$$

Every category contributes a coin-flip factor whether bought or not; a bought category contributes three more. Reading Eq. 21 left to right is exactly the sampler of Section 6, stage by stage.

4.6 What is fitted, and the three ways it differs from Eq. 21

Training does not maximise $\log P(\mathcal{B})$. It minimises a weighted sum of four losses (Section 5.1), and three differences are worth naming because they change what the reported numbers mean.

(i) The choice term is conditional on the rest of the cart. Eq. 11 draws k_c products with no reference to the other categories. Training instead scores each purchased row against the whole rest of the basket, through $\bar{\alpha}_j(S)$, and so maximises $\sum_j \log P(j \mid \mathcal{B}\setminus j)$. Eq. 19 shows those conditionals belong to a genuine distribution, which is what licenses the sampler; but they are not the factor $P(J_c \mid k_c)$ in Eq. 21.

(ii) The two choice sets differ. Eq. 11 normalises over the category's stocked items. Training normalises over the true item plus negatives drawn from the *whole catalogue*, while generation draws within a category (`27_nested_basket.py:423` against `28_nested_counterfactual.py:436`). The term is therefore fitted as a catalogue-wide choice and used as a within-category one. Section 8.13 finds that drawing negatives from the true item's own category improves generated co-occurrence sharply. Closing this gap is one plausible reason; we have not separated it from the other.

(iii) The terms are separately averaged and weighted. Each loss is a mean over its own sample — purchase rows, sampled categories, bought categories — then multiplied by w_q, w_c, w_b. Eq. 21 implies particular relative weights; the objective does not reproduce them.

Consequence. The model has a well-defined distribution over baskets, Eq. 21, and Section 8.10 tests it by sampling from it and comparing against real baskets. It has no tractable likelihood for a whole basket: no number in this paper is $\log P(\mathcal{B})$ on held-out data. Every likelihood reported is the conditional in (i).

5 Inference

We fit by stochastic gradient descent on a weighted sum of the four terms. This section gives the objective, the three sampling schemes it depends on, and the computational cost. Code: `scripts/27_nested_basket.py`.

5.1 Objective

$$\mathcal{L} = \mathcal{L}_{\text{item}} + w_q\mathcal{L}_{\text{qty}} + w_c\mathcal{L}_{\text{inc}} + w_b\mathcal{L}_{\text{brd}} + \frac{1}{B}\Big(\ell_2\|\Theta_{\text{repr}}\|^2 + \ell_2^p\|\Theta_{\text{price}}\|^2 + \ell_2^W\|W\|^2\Big). \tag{22}$$

The penalties are separated deliberately. Θ_{repr} covers $\alpha, \theta, \eta, \mu, \delta, \zeta, \xi$ and the incidence embeddings; Θ_{price} covers only $\gamma, \beta, \gamma^q, \beta^q$. Shrinking the price block at the representation block's rate biases the elasticity toward zero, and the elasticity is the quantity being estimated. W needs a third penalty for a reason Section 8.11 makes concrete: left unregularised it absorbs signal that belongs to $\kappa \cdot IV$, which is the only channel price reaches incidence through.

Optimiser Adam, learning rate 0.005 annealed by cosine to 0.05× over 12,000 iterations. A batch is 192 baskets expanded to all their purchase rows. Validation runs every 6,000 iterations and the checkpoint is kept on $\ell^{\text{val}}_{\text{item}} - w_q\text{NLL}^{\text{val}}_{\text{qty}} - w_c\text{NLL}^{\text{val}}_{\text{inc}}$; breadth is trained but has no held-out scalar and so does not enter model selection.

5.2 Negative sampling

The item softmax over 5,455 items is intractable per row, so we sample K negatives. The default distribution is $p(j) \propto \max(\text{count}_{\text{train}}(j), 1)^{0.75}$ over the whole catalogue, and negatives are drawn unconditionally and then masked by availability — so the *effective* number of competitors is below K and varies by store.

This choice turns out to matter more than any modelling decision in the paper. Section 8.13 shows that a decoy drawn this way is almost always separable by $\lambda_j + \theta_i^\top \alpha_j$ alone, so the loss asks "is this item plausible at all" and never "does this item fit *this* basket". We therefore also study drawing a fraction f of negatives from the true item's own category, which is SHOPPER's `-nsFreq` ≥ 2 option.

5.3 The inclusive value is estimated, not summed

IV_{ict} is a log-sum-exp over a category's stocked items. Categories are padded to the largest (225 items) although the median has 15, so a dense evaluation spends most of its work on padding. Write n_c for the number of items of category c the trip's store carries. We sample $m = 32$ of them uniformly with replacement and rescale:

$$\widehat{IV} = \log\Big(\sum_{s=1}^{m} e^{u_{j_s}}\Big) + \log\frac{n_c}{m}. \tag{23}$$

The *sum* inside is unbiased, since $\mathbb{E}[\frac{n_c}{m}\sum_s e^{u_{j_s}}] = \frac{n_c}{m} \cdot m \cdot \frac{1}{n_c}\sum_j e^{u_j} = \sum_j e^{u_j}$. Its *logarithm* is not: by Jensen $\mathbb{E}[\log X] \leq \log \mathbb{E}[X]$, so $\widehat{IV}$ is biased slightly low. The bias is absorbed into the category intercept c_c^0 during training, and generation uses the same estimator, so the two remain consistent.

A frozen reference. $\overline{IV}_c$ in Eq. 9 is fixed once, at iteration 500, to the mean IV over a *uniform* category sample. Any fixed constant is absorbed by c_c^0, so freezing does not change the fit; it guarantees that training and generation subtract the same thing. Taking the reference from the training batches instead uses whatever category mix those batches had, and if that mix differs from the population the generated baskets come out systematically wrong.

5.4 Category sampling for the incidence head

Incidence is a Bernoulli over 188 categories per trip, of which only 6.12 are bought. We sample 16 categories per trip **uniformly**. Case-control sampling (a few bought, a few not) is tempting at a 3.25% base rate, and biases the fit two ways: the intercept, which the standard $\log(\pi_1/\pi_0)$ offset corrects, and the term $\kappa_c(IV - \overline{IV}_c)$, whose mean differs between sample and population because bought categories have higher inclusive values — which the offset does not touch. Uniform sampling removes both at source and needs no correction.

5.5 Computational cost

run	iterations	wall time	ms/iter
default	12,000	10.8 min	53.8

Two lookups dominate and both are sparse and high-cardinality: household state and store price. Materialising (household $\times$ day $\times$ sub-commodity) is about 32 million rows and a per-sample dictionary is about 35 million hits per epoch. Both are instead stored once as a globally sorted key array with key = group $\cdot$ stride + index, so a whole batch resolves in one `np.searchsorted`; the stride exceeds any index so ordering never crosses a group boundary. That search is still roughly 40% of a step, and is avoidable by deduplicating repeated candidates within a batch, which we have not done.

Scalability. Cost is independent of catalogue size (negative sampling fixes the candidate count), of the number of households (only the embedding table grows), and of the number of baskets per step. Categories are capped at 16 sampled per trip and items per category at 32 by the sampled inclusive value. Nothing is quadratic in items or households.

6 Sampling

Generation is a harder requirement than prediction and is where most of this paper's findings came from. We state precisely what the model gives us and what it does not.

What we have. Conditionals: $P(\text{item} \mid \text{rest of basket})$ from the item head, $P(\text{category bought})$ from the incidence head, and count distributions from the quantity and breadth heads. **What we do not have** is a normalised joint $P(S)$: computing it needs a partition function over all size-n baskets, which is intractable. Eq. 19 shows the item head's conditionals are those of $P(S) \propto e^{E(S)}$ at fixed n, so we can *sample* that joint without ever evaluating it.

Stage 4 holds basket size and category composition fixed, which is the move Eq. 19 is defined over. Stage 1's optional sweep is the same argument one level up: the category indicator is a C-dimensional binary field whose single-site conditionals the incidence head supplies.

Algorithm 1: Basket generation

Input: household i, day t, week w, store s

Output: a basket of items with unit counts

Stage 1 — categories. For each c, compute IV_{ict} and draw $\mathbb{1}\{c \in B\} \sim \text{Bernoulli}(\sigma(\text{logit}_{ict}))$;

if *the category interaction is on* **then**

  Gibbs sweep the C-dimensional indicator: for each c, recompute logit_{ict} given the current $B \setminus c$ and redraw;

end

Stage 2 — how many. For each $c \in B$ draw $k_c = 1 + \text{Poisson}(e^{z^b_{ic}})$, capped at the number the store stocks;

Stage 3 — which. For each $c \in B$, draw k_c distinct items from $\text{softmax}_j u_{ijt}$ over that category's stocked items, with context zero (no basket exists yet);

Stage 4 — refine. Gibbs sweeps: for each filled slot, remove its item, recompute $\bar{\alpha}$ from the current draft, and redraw from that slot's own category;

Stage 5 — units. For each chosen item draw $1 + \text{Poisson}(e^{z_{ijt}})$;

A sequential alternative. If the model is trained with a random-prefix context — one permutation per basket, each row seeing only the items before it, which is SHOPPER's unordered likelihood (its Eq. 6) — then Stages 3–4 collapse into one pass that places items one at a time with $\bar{\alpha}$ the running mean of what is already placed. Training and generation then use identical conditionals and no Gibbs is needed. Section 8.12 evaluates this.

What generation does not carry. With the context zeroed at Stage 3 and no Stage 4, generated baskets reproduce marginals only. This is not a detail: it is what Section 8.10 measures and Section 8.11 spends its length on.

7 From measurement to model

what Section 3 showed	what the model does
25% of the units response is existing buyers taking more	quantity term with its own price coefficient
73% of the units response is entering a category at all	the nest, with κ_c estimated rather than assumed
1.284 distinct items per purchased category; 22.3% of rows buy more than one unit	breadth term; units kept as counts, never binarised
Taste is 1.66× more self-similar within household than across	per-household θ_i
Per-household price slopes have split-half reliability +0.236	price sensitivity low-rank $\gamma_i^\top \beta_j$, not free per pair
Median store carries only 63% of the catalogue	availability mask: unstocked items leave the choice set
15.8% of store-item-weeks differ from the chain price	store-level price deviation where observed
Week effects carry a confound between price and demand	$\mu_j^\top \delta_w$ seasonality control
Promotion weeks are 3.4× steeper than ordinary variation	*not addressed*; Section 10

8 Empirical study

8.1 Protocol, and what is comparable

All numbers are on held-out test weeks. The seed spread on item log-likelihood is **0.0032** nats, so gaps larger than about 0.01 are real.

Two comparability rules are applied throughout, because breaking them silently is the easiest way to report a false gain. **Item log-likelihood is conditional on the rest of the basket** — it is a fill-in-the-blank score, not "predict the basket". And **it is only comparable across models scored against the same negative distribution**; where a variant changes that distribution we omit the column and rely on the generator checks, which use no negatives at all.

8.2 Do the equations match the code?

Fifteen checks compare each printed formula against the fitted model, exactly where the paper states an identity and by finite difference where it states a derivative (`scripts/33_verify_equations.py`). All pass: Eq. 7 term by term; the context formula; the swap identity Eq. 19; the Poisson likelihood; κ = softplus; the implied breadth λ/p; the three elasticity derivatives of Section 8.8; and both claims about the sampled inclusive value in Section 5.3.

8.3 Ablations

model	item log-lik	top-1	cost of removing
full model	−1.9927	0.378	—
seed replicate	−1.9895	0.379	—
no store	−2.1820	0.355	**0.189**
no interaction	−2.1074	0.347	**0.115**
no household state	−2.0684	0.360	**0.076**
prices scrambled	−2.0416	0.362	0.049
availability mask only	−1.9932	0.375	0.001
no breadth	−1.9927	0.378	0.000
no quantity	−1.9925	0.378	0.000
no nest	−1.9877	0.380	−0.005

Three of the four terms cost nothing on item ranking and the nest is marginally better without. **This is the design working, not failing.** The item term ranks items; the nest, quantity and breadth terms answer questions it never asks, and they share only u_{ijt}. A term that improved item ranking *because* it also modelled incidence would mean the two were entangled. Each must therefore be scored on its own quantity, and the breadth term is the case in point: worth 0.000 nats here, and without it generated baskets hold 6.94 items against a real 8.18.

Stores, split honestly. Of the 0.189 nats attributed to store information, 99.7% is the availability mask alone, which merely shrinks the choice set. Modelling stores was a correctness fix, not an information gain, and it makes item log-likelihood incomparable to models that score unstocked items as rejected alternatives.

8.4 Against heuristic baselines

Every model is scored through one scorer on identical candidate sets — same positives, same negatives, same availability mask — on 51,167 held-out purchases with 20 negatives each. Baseline weights are tuned on validation.

model	log-lik	top-1
nested	−2.0011	**0.371**
nested, no interaction	−2.1195	0.342
household repeat-purchase	−2.3428	0.285
popularity	−2.7375	0.091
random	−2.7378	0.067
household + co-occurrence	−3.0526	0.182
item–item co-occurrence	−3.0655	0.078

The strongest baseline is not popularity but household repeat-purchase; grocery is repetitive and most of what any model can predict is that people rebuy what they always rebuy. The real margin is +0.342 **nats and 8.5 points of top-1**. Two results cut against the model: popularity is indistinguishable from random, because negatives are drawn popularity-weighted, so "we beat popularity" is weaker than it sounds; and raw item–item co-occurrence scores *below* random, so the signal exists but counting cannot exploit it against popularity-matched decoys.

8.5 Against learned baselines, and where prices move

The comparisons in Section 8.4 are against heuristics. Three are weak by construction and the fourth — raw item–item co-occurrence — scores *below* random, which flatters our model: it is a counting rule standing in for what should be a learned item–item model. SHOPPER [1] compares against two latent-factor models instead, and they are the right competitors. **HPF** [3] factorises the household $\times$ item matrix: it has the household half of our model and neither interaction nor price. **B-Emb** [4] is a skip-gram with negative sampling over within-basket pairs: the interaction half, and neither household nor price. Between them they bracket our specification. Both temperatures are tuned on validation.

SHOPPER also runs a second evaluation we had not: scoring only those held-out purchases whose price sits furthest from its item's own average. That is where a model claiming to answer price counterfactuals has to prove itself, and it is a harder subset than an average trip. We report both.

model	full test set log-lik	top-1	top 10% price variation log-lik	top-1	change
HPF (user $\times$ item) [3]	-2.6028	0.163	-2.6260	0.152	-0.023
B-Emb (item–item) [4]	-2.4684	0.239	-2.4920	0.228	-0.024
nested	-1.9971	0.376	-1.8764	0.429	$+0.121$
nested_both	-1.9908	0.380	-1.8705	0.430	$+0.120$
nested_hardneg	-2.0867	0.358	-1.9790	0.406	$+0.108$

Two things follow, one of which corrects Section 8.4.

B-Emb is the strongest baseline, at -2.4684. It beats household repeat-purchase and it beats raw co-occurrence by more than half a nat. The margin our model actually earns over a serious competitor is therefore $+0.471$ nats, not the larger figure the co-occurrence comparison implied. Measuring a learned interaction against a counting rule was measuring against a straw man.

The model improves where prices move and the baselines degrade. On the top price-variation decile the nested model gains $+0.121$ nats and its top-1 rises from 0.376 to 0.429, while HPF and B-Emb each lose about 0.02. Neither baseline has a price term, so on the subset where price is doing the work they fall back; ours does not. This is stronger evidence than the placebo of Section 8.7: the placebo shows the coefficient dies when prices are destroyed, whereas this shows the model *gaining* exactly where the mechanism should bite.

8.6 Deployment: recommending without a basket

Every item log-likelihood in this paper is conditional on the rest of the basket. A recommender does not have that. At the moment a recommendation is made the cart holds $0, 1, 2, \ldots$ items, never all of them but one. So we score the held-out item given a random k-item prefix of its own basket (`scripts/35_context_ablation.py`).

model / repair	$k=0$	1	2	4	8	full
	items already in the cart					
nested, raw	−2.1385	−2.1832	−2.0823	−2.0299	−2.0068	−1.9971
norm-matched	−2.1385	−2.0930	−2.0703	−2.0463	−2.0261	−1.9971
nested_both, raw	−2.2483	−3.0861	−2.5009	−2.2084	−2.0641	−1.9908
norm-matched	−2.2483	−2.2786	−2.1860	−2.1086	−2.0387	−1.9908
nested_hardneg, raw	−2.3096	−2.9013	−2.4586	−2.2481	−2.1363	−2.0867
norm-matched	−2.3096	−2.3105	−2.2390	−2.1836	−2.1248	−2.0867

The model is usable without a basket. At $k = 0$ it scores −2.1385 against −2.5230 for tuned household repeat-purchase, which needs no basket at all. Not knowing the basket costs 0.141 nats against the full-basket score, so the headline number does overstate the deployment setting, by about 7% of the model's margin over popularity. A model *trained* without the context reaches −2.2483 at $k = 0$, so the mismatch penalty from deploying a context-trained model without context is only 0.020 nats: one model suffices.

One item in the cart is worse than none. Every model degrades at $k = 1$, and `nested_both` catastrophically, to −3.0861 — a full nat below cold start. The cause is a norm mismatch, not a modelling error. Averaging shrinks a vector, so $\|\bar{\alpha}\|$ over a full basket is much smaller than over a single item:

$\|\bar{\alpha}\|$ relative to training	$k=1$	$k=2$	$k=4$	$k=8$
nested	2.31×	1.74×	1.36×	1.15×

At $k = 1$ the context is 2.31× the norm training ever produced, and `nested_both` multiplies that by its learnable scale of 5.52, which is why it fails hardest. **Rescaling $\bar{\alpha}$ to the training norm repairs it** without retraining: `nested_both` recovers 0.81 nats at $k = 1$, and for every model the repaired $k = 1$ score is better than cold start rather than worse. Simply ignoring a small cart also avoids the failure but is strictly weaker, since it discards the information a one- or two-item cart carries.

Two caveats. Norm-matching costs a little at large k for the tied model (−2.0068 raw against −2.0261 matched at $k = 8$), because forcing a fixed norm discards the genuine signal that a large cart's context is legitimately smaller; a k-dependent shrinkage would likely dominate both and we have not tested it. And the variants recommended for *generation* are the worst for *recommendation*: plain `nested` leads at every cart size below 8.

8.7 Is the price response causal?

We refit the whole model on a price panel scrambled before fitting, including the store-level deviations — leaving those real would leak genuine prices back in.

model	price coef.	quantity price coef.	κ
fitted	+0.794	+0.134	0.663
seed replicate	+0.794	+0.109	0.674
prices scrambled	−0.000	−0.000	0.675

The placebo retains 0.0% of the coefficient and costs 0.049 nats of ranking while leaving κ and the incidence likelihood untouched: prices scramble, category structure does not. This complements the design-based evidence in Section 3, where demand does not ramp in the weeks before a cut.

8.8 Where a price cut goes

From the softmax and the Poisson–multinomial,

$$\frac{\partial \log \pi_j}{\partial \log p_j} = -(\gamma_i^\top \beta_j)(1-\pi_j), \qquad \kappa_c \frac{\partial IV}{\partial \log p_j} = -\kappa_c(\gamma_i^\top \beta_j)\pi_j, \tag{24}$$

$$\frac{\partial \log \mathbb{E}[\text{units}]}{\partial \log p_j} = -(\gamma_i^{q\top} \beta_j^q)\frac{\lambda}{1+\lambda}, \qquad \lambda = e^{z_{ijt}}, \tag{25}$$

where λ is the quantity head's Poisson rate, so $\mathbb{E}[\text{units}] = 1+\lambda$. The first two are exact derivatives of the same softmax, which is why they carry $(1-\pi_j)$ and π_j and why they combine to $-(\gamma^\top\beta)[1-\pi_j(1-\kappa_c)]$. On 12,724 held-out purchases: allocation -0.991 (83%), incidence -0.058 (5%), quantity -0.139 (12%), total -1.188. Shares are computed from means because medians do not decompose additively.

Mostly a price cut moves share. The quantity margin is 12% — unreachable for a binary-purchase model, and corroborated by the model-free -0.235 of Section 3.1. Read κ with care: it moved with the incidence sampler across three versions before settling, so it is weakly identified and no argument here rests on its exact value.

8.9 Do the embeddings recover product structure?

Sub-commodity labels are never shown to the model.

	kNN purity	× chance	AUC
fitted α	0.2976	69.6×	0.8222
no interaction	0.2119	49.5×	0.7267
random control	0.0037	0.9×	0.499
popularity control	0.0038	0.9×	0.510

The interaction is the only component besides the availability mask that materially moves the embedding. On a 409-item universe shared with the nested-factorization model of [2], α reaches 12.1× chance and AUC 0.805 against 1.0× and 0.379; that model's below-chance AUC is a mechanism, not noise, since its within-category softmax makes close substitutes compete and pushes them apart.

8.10 Generation: does the distribution match?

Comparing three means is not enough, and comparing $\alpha^\top\alpha$ would be circular — the model is trained to make co-purchased items have large $\alpha^\top\alpha$. Every check below is **model-free** (item ids and counts only) and **distributional**. Generated trips are matched to real ones on household, day, week and store, so any gap is the model's. All 24,768 test baskets; `scripts/34_generator_eval.py`.

1. Shape. Total variation distance, $\frac{1}{2}\sum_x |P_{\text{real}}(x) - P_{\text{gen}}(x)|$.

	real	generated	TVD
items per basket	8.18	9.11	**0.30**
categories per basket	6.38	6.85	**0.30**
units per basket	11.06	12.16	**0.29**
distinct items per category	1.28	1.33	0.06
units per line	1.35	1.34	0.02

The means agree and the distributions do not; real baskets are far more skewed. The two quantities the model addresses *directly* — breadth and units per line — match closely, while the three that emerge from composing them do not.

2. Item marginals. Per-item purchase rate across all 5,455 items: Spearman +0.406, with 0.1% of real items never generated. No mode collapse.

3. Co-occurrence lift. $\text{lift}(j,k) = P(j,k \text{ together})/(P(j)P(k))$ for the commonest items: real 3.48, generated 1.61, per-pair Spearman −0.003.

4. Held-out labels. Share of within-basket pairs sharing a sub-commodity: real 0.0646, generated 0.0593.

5. Price response from generated data. Rebuilding the item × week panel from synthetic baskets and applying Eq. 5: real −0.6480, generated −0.5669, i.e. **87%** of the real response. This is the property a pricing policy would be learned from and it is the best of the five checks.

8.11 The co-occurrence failure, and eleven attempts on it

Check 3 is the failure. Three causes were identified by measurement.

Cause 1: the wrong level. The item-level Gibbs sweep resamples within a category, because that is the move Eq. 19 defines. Splitting generated pairs by whether the sweep can reach them: same-category pairs reach a per-pair Spearman of +0.290, cross-category pairs −0.094. **95% of item pairs are cross-category**, and category choice was C independent Bernoulli draws.

Cause 2: dilution. $\bar{\alpha}$ divides by $n-1$, so a partner worth $\alpha_j^\top \alpha_k = 6$ contributes 6/7 in an eight-item basket. Given a free scalar the model sets it to 5.5, close to the average $n-1$.

Cause 3: tied embeddings encode similarity. $\alpha^\top \alpha$ averages 4.34 within a sub-commodity and 0.47 across categories. A *tied* category-level context fitted its scale to −3.0: the model used it for basket-size competition because it could not use it for complementarity. At category level the escape is arithmetic — $C = 188$ makes a full pairwise matrix 17,578 parameters, impossible at item level ($J^2 \approx 30\text{M}$).

variant	item log-lik	inc NLL	κ	lift	same-sub	price
base	−1.9927	0.1104	0.663	43%	57%	82%
+ learnable scale	−1.9966	0.1116	0.658	44%	80%	80%
+ category context	−1.9970	0.1075	0.828	44%	89%	84%
+ pair matrix, weak L_2	−1.9971	0.0987	0.597	49%	121%	70%
+ pair matrix, L_2=0.3	−1.9966	0.1051	0.665	44%	86%	81%
both, regularised	−1.9970	0.1012	0.815	46%	92%	87%

Unregularised, the pair matrix is the best model of category co-occurrence and drags the price response to 70%, with κ falling to 0.597 — price reaches incidence only through $\kappa \cdot IV$, so anything explaining co-occurrence more cheaply takes that job. Penalising it separately restores both. Stacking the tied context, which pushes κ up, with the matrix, which pushes it down, holds κ at 0.815 and improves four of five checks at a cost inside the seed spread.

8.12 SHOPPER's asymmetric interaction and sequential likelihood

Cause 3 above says complementarity is not representable by a symmetric tied form. SHOPPER is built on exactly the mechanism that diagnosis calls for: ρ untied from α, asymmetric, free to go negative. We fitted it, and also its sequential likelihood.

variant	item log-lik	lift	Spearman	same-sub
tied (default)	−1.9970	46%	−0.003	92%
untied ρ	−2.0550	47%	−0.005	62%
untied ρ, sequential	−2.0645	47%	+0.010	57%
one-vs-each loss	−2.9677	47%	+0.001	86%

Neither moved the number they were built for. The freedom was used — $\cos(\rho_j, \alpha_j) = +0.31$, so ρ found a substantially different direction — but it did not find real complements. Both cost item ranking and both lose the same-sub gain, which follows: with ρ carrying the interaction, α no longer has to encode co-purchase structure. **This refutes the Cause 3 diagnosis**: the geometry was never the binding constraint.

We also fitted SHOPPER's one-vs-each bound ($\log \mathrm{softmax}_i \geq \sum_{i'} \log \sigma(u_i - u_{i'})$), its default objective, at 50 negatives. Spearman +0.001, and item log-likelihood −2.9677 — a full nat worse than baseline on a comparable measure. Changing the loss functional form did not help either.

8.13 The resolution: harder negatives

Eleven changes to the model left the per-pair correlation at zero. One change to the *negative distribution* moved it. Replacing a fraction f of negatives with items from the true item's own category removes popularity and broad taste as discriminators and leaves the basket context as the only signal that can separate them.

in-category f	inc NLL	κ	lift	Spearman	z	same-sub	price
0	0.1012	0.815	46%	−0.003	−0.4	92%	87%
0.25	0.1044	0.850	50%	+0.028	+4.1	84%	88%
0.50	0.1037	0.825	54%	+0.087	+13.1	81%	89%
0.75	0.1068	0.781	54%	+0.091	+14.1	88%	85%
1.00	0.1062	0.626	56%	+0.081	+13.4	75%	76%
0.50, 50 neg.	0.1043	0.827	52%	+0.063	+9.5	81%	84%

It is a dose-response curve, which is what distinguishes a mechanism from a lucky setting: Spearman rises monotonically to $f = 0.75$ and reaches $z = +14.1$ on roughly 22,000 pairs, against $z = -0.4$ at $f = 0$. Co-occurrence lift rises 46% to 54%, and per-item marginals improve monotonically throughout.

Two limits. *Past* 0.75 *it costs more than it buys*: at $f = 1$ the model never sees an implausible decoy, κ collapses to 0.626 and the generated price response falls to 76% — the crowding-out signature again. And *more negatives is not harder negatives*: at the same $f = 0.5$, going from 20 to 50 negatives loses on every measure, although SHOPPER defaults to 50.

Recommendation. $f = 0.5$ if the generator is for pricing policy (price response 89%, the best in this paper); $f = 0.75$ if it is for basket composition (best pairwise ranking and same-sub 88%).

9 Related work

SHOPPER [1] models a basket as a sequence of choices with an asymmetric interaction $\rho_c^\top \bar{\alpha}$, per-customer price sensitivity as a low-rank bilinear form, and week effects for seasonality. We inherit all three. We differ in adding an explicit incidence stage with an estimated nesting coefficient, in modelling unit counts rather than binary purchase, and in conditioning on the store; and we replace its variational inference with maximum *a posteriori* estimation by SGD, which loses uncertainty quantification and gains an order of magnitude in fitting time. Section 8.12 reports that its asymmetric interaction did not improve generation on this dataset, and Section 8.13 that its group-biased negative sampling did.

Section 8.5 compares against the two latent-factor baselines SHOPPER uses — hierarchical Poisson factorization [3] and exponential family embeddings [4] — and adopts its evaluation on high price-variation purchases.

The nested factorization of [2] supplies the nesting coefficient's interpretation and the argument that week effects suffice to identify the price effect. Its within-category softmax requires at most one item per category per trip, which 68.2% of baskets here violate, and Section 8.9 shows the same construction pushes close substitutes apart in embedding space.

10 Discussion

What the model does well. It separates three margins that are economically distinct and estimates their coupling; its price response survives a structural placebo; and synthetic baskets carry 87% of the real price elasticity, which is the property that matters if the generator is to be used for policy learning.

What it does not do.

1. **Per-pair co-occurrence is only partly learned.** At best the per-pair rank correlation is $+0.091$ — real ($z = +14.1$) but far from strong. Generated baskets have roughly the right amount of coherence and only weakly the right pattern.

2. **Promotion weeks are steeper than a single price coefficient allows.** Section 3 measures an implied -2.83 in the cut week against -0.792 on ordinary variation. Display and feature are not observed, so one coefficient averages two regimes.

3. **Basket-size distributions are wrong in shape** even where means agree.

4. **There is no held-out joint likelihood.** The model has a well-defined generative distribution over baskets (Eq. 21) but no tractable likelihood for one: the item stage is fitted as a conditional, its softmax is normalised over a different set than the generative one, and the four terms are weighted rather than multiplied. Every likelihood in this paper is the *conditional* of Section 4.5, never $\log P(\mathcal{B})$. Generation, evaluated in Section 8.10, is the only test of the joint.

5. **The item head is trained on one choice set and used on another.** Negatives come from the whole catalogue; generation draws within a category (Section 4.5). Section 8.13 shows that closing this gap improves co-occurrence sharply, but we did not separate that explanation from the one we gave.

6. κ **is weakly identified**, having moved with the incidence sampler rather than the data before settling.

7. **Partial baskets need a correction at serving time.** The interaction term is trained on a context averaged over a whole basket, and a one-item cart produces a vector 2.31 times that norm. Without rescaling, a cart with one item is worse than an empty one (Section 8.6).

8. **No uncertainty quantification.** We report point estimates; the seed spread bounds run-to-run noise but is not a posterior.

A methodological lesson. The most useful finding in this paper was not a model. Eleven changes to the interaction term — including the one a well-supported diagnosis predicted would work — moved the target quantity by nothing, and a change to the negative sampling distribution moved it by fourteen standard errors. The distinguishing evidence was a dose-response curve, and none of it would have been visible without a model-free distributional evaluation of the generator. Where a model is meant to generate data, it should be judged on the data it generates.

References

[1] F. J. R. Ruiz, S. Athey and D. M. Blei. *SHOPPER: A Probabilistic Model of Consumer Choice with Substitutes and Complements*. Annals of Applied Statistics, 2020. arXiv:1711.03560.

[2] S. Donnelly, F. J. R. Ruiz, D. M. Blei and S. Athey. *Counterfactual Inference for Consumer Choice Across Many Product Categories*. Quantitative Marketing and Economics, 2021.

[3] P. Gopalan, J. M. Hofman and D. M. Blei. *Scalable Recommendation with Hierarchical Poisson Factorization*. UAI, 2015.

[4] M. Rudolph, F. J. R. Ruiz, S. Mandt and D. M. Blei. *Exponential Family Embeddings*. NeurIPS, 2016.

[5] M. K. Titsias. *One-vs-Each Approximation to Softmax for Large Scale Classification*. NeurIPS, 2016.